

PROBLEM

Pre-DI: The Problem

WHY TIGHTLY-COUPLED CODE BECOMES BRITTLE AND HARD TO TEST

1 Tightly-Coupled Code

ANTI-PATTERN

```
class OrderService {  
    private StripeGateway payment = new StripeGateway(); // hardcoded!  
    private FileLogger logger = new FileLogger(); // hardcoded!  
  
    void placeOrder(Order order) {  
        logger.log("Placing order " + order.id);  
        payment.charge(order.total);  
    }  
}
```

WHY THIS HURTS

1. Hardcoded.

OrderService is welded to StripeGateway and FileLogger by name — switching to PayPal means editing OrderService itself.

2. Untestable.

A unit test for placeOrder() always hits the real Stripe API and writes a real log file. There's no way to substitute a fake.

3. Backwards dependency.

The high-level OrderService depends directly on low-level concrete classes, not on abstractions.

4. Duplicated setup.

Every class that needs a payment gateway repeats the same new StripeGateway() line.

THE SETUP FOR WHAT'S NEXT

This is exactly the problem DI solves — instead of a class creating its own dependencies, something else hands them in from outside.